Third-Party Integrations

Payments, identity, messaging, maps, Google and Azure dependencies.

Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository: rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @ 3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Payments, identity, messaging, maps, Google and Azure dependencies. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

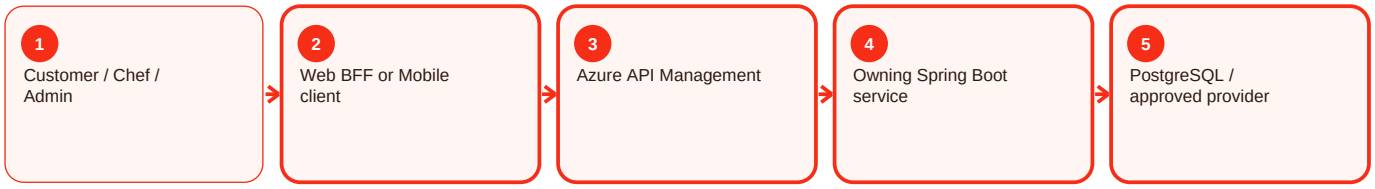
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Provider adapter pattern - Overview and responsibility

Current implementation

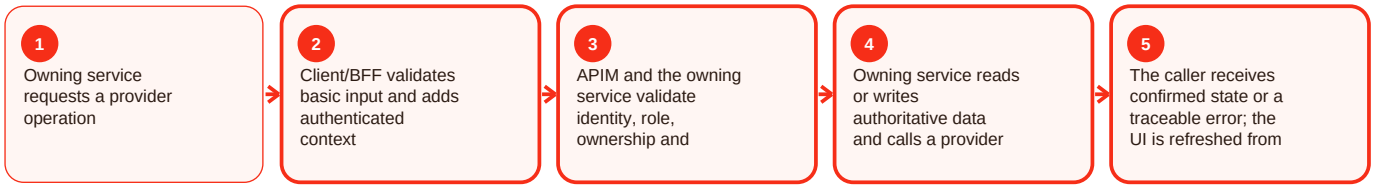
Summary

Provider adapter pattern is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For provider adapter pattern, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Provider adapter pattern - Functional behavior and data

Current implementation

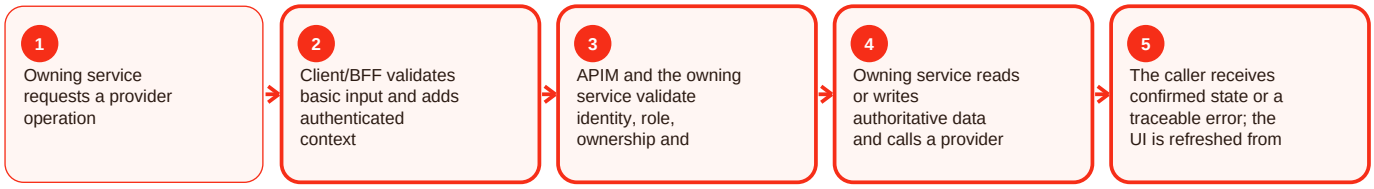
Summary

Provider adapter pattern is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For provider adapter pattern, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Provider adapter pattern - Operations, errors and deployment

Current implementation

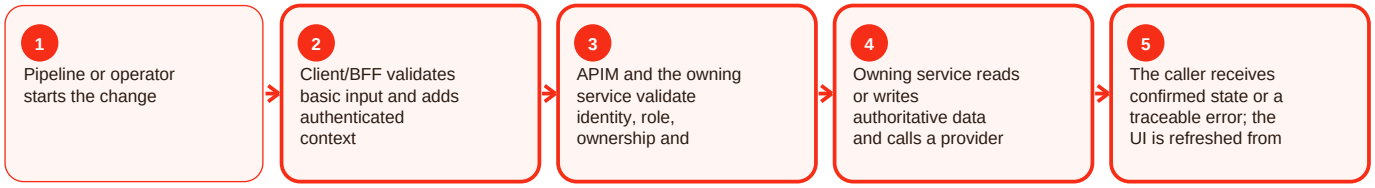
Summary

Provider adapter pattern is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For provider adapter pattern, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Safe stubs - Overview and responsibility

Current implementation

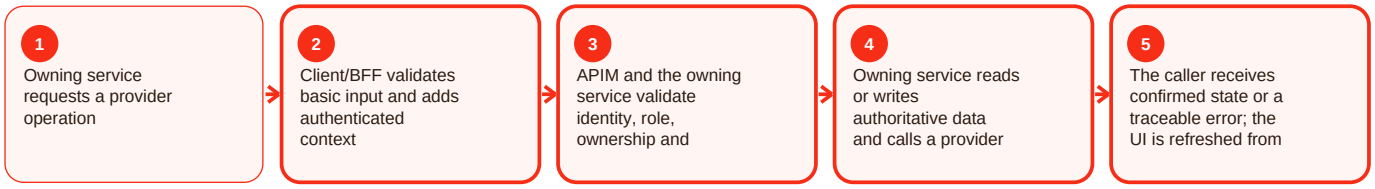
Summary

Safe stubs is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For safe stubs, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Safe stubs - Functional behavior and data

Current implementation

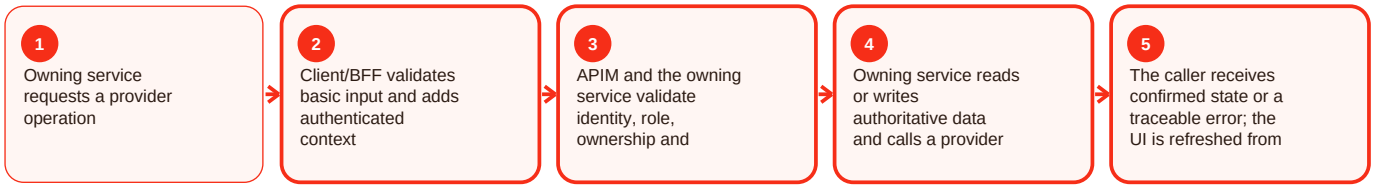
Summary

Safe stubs is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For safe stubs, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Safe stubs - Operations, errors and deployment

Current implementation

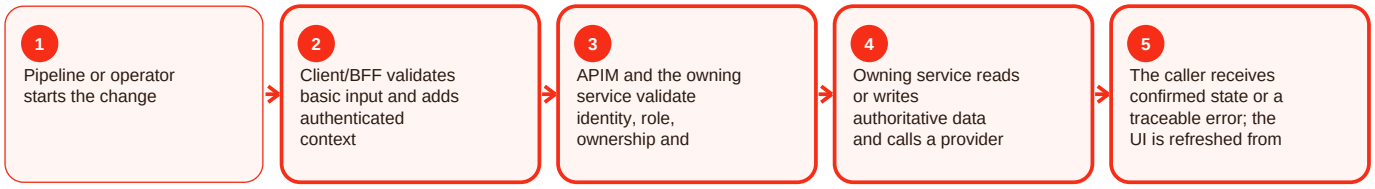
Summary

Safe stubs is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For safe stubs, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cashfree - Overview and responsibility

Current implementation

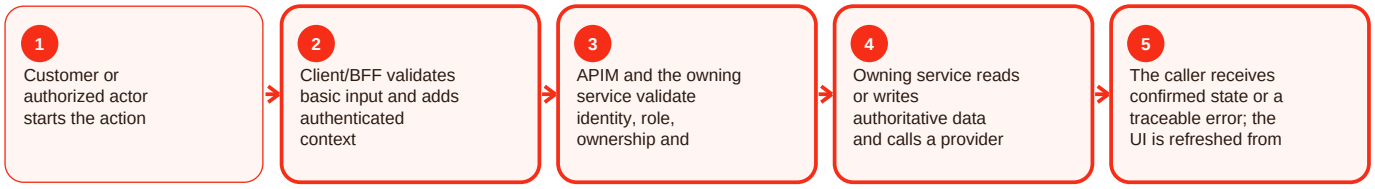
Summary

Cashfree is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cashfree - Functional behavior and data

Current implementation

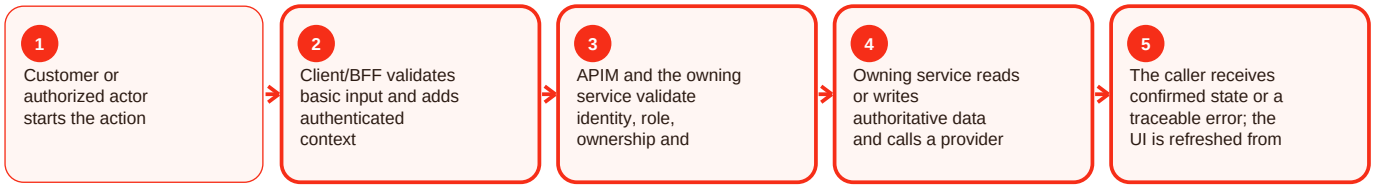
Summary

Cashfree is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cashfree - Operations, errors and deployment

Current implementation

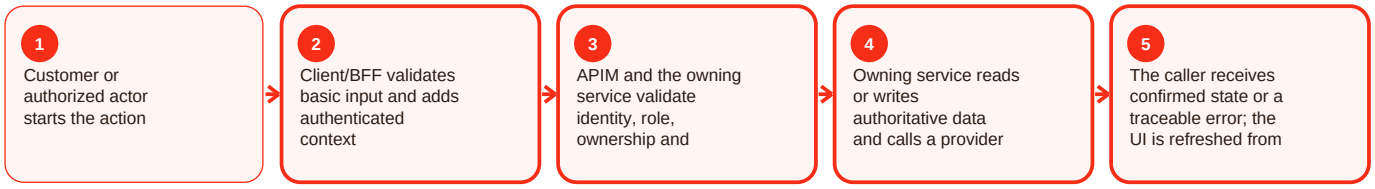
Summary

Cashfree is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Razorpay webhook hmac - Overview and responsibility

Current implementation

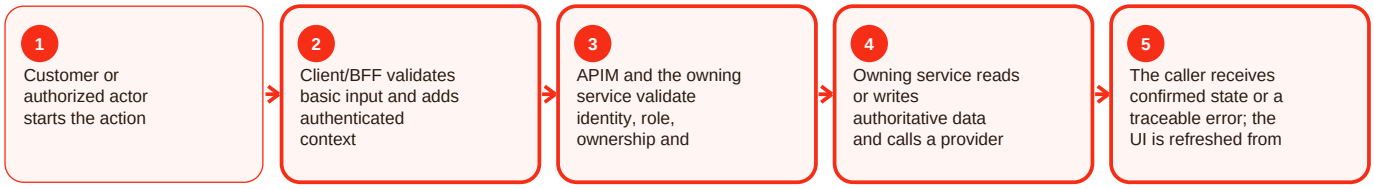
Summary

Razorpay webhook hmac is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay webhook HMAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Razorpay webhook hmac - Functional behavior and data

Current implementation

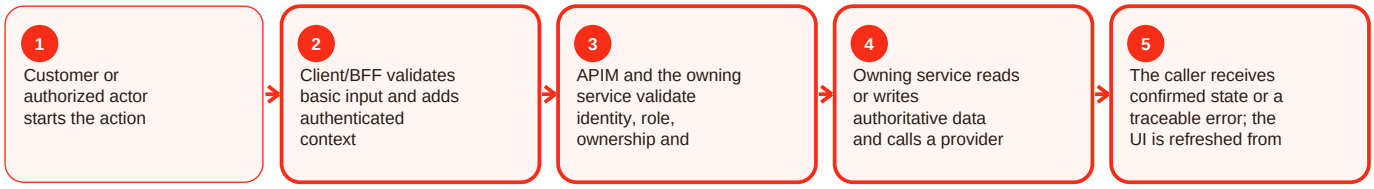
Summary

Razorpay webhook hmac is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay webhook HMAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Razorpay webhook hmac - Operations, errors and deployment

Current implementation

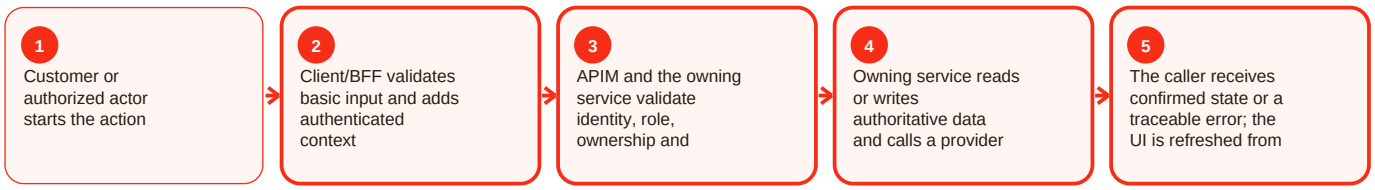
Summary

Razorpay webhook hmac is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay webhook HMAC, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Razorpay payment links - Overview and responsibility

Current implementation

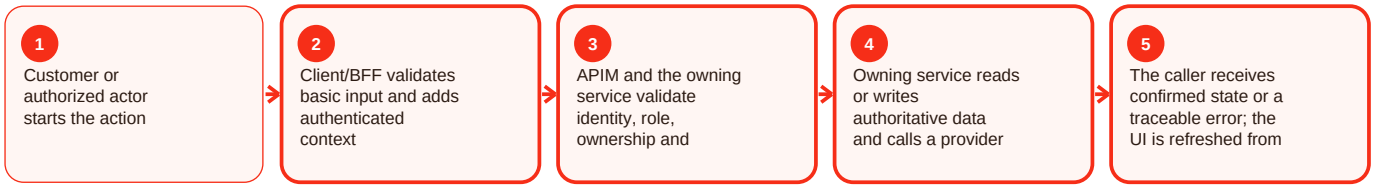
Summary

Razorpay payment links is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay payment links, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Razorpay payment links - Functional behavior and data

Current implementation

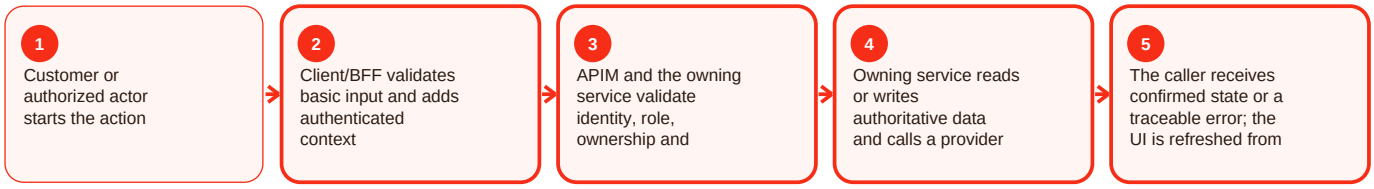
Summary

Razorpay payment links is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay payment links, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Razorpay payment links - Operations, errors and deployment

Current implementation

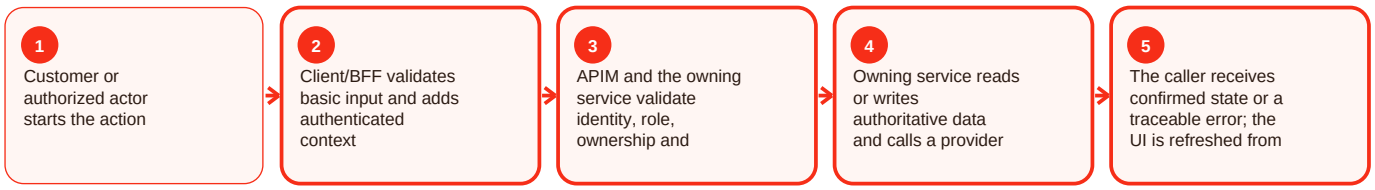
Summary

Razorpay payment links is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Razorpay payment links, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refund / provider state - Overview and responsibility

Current implementation

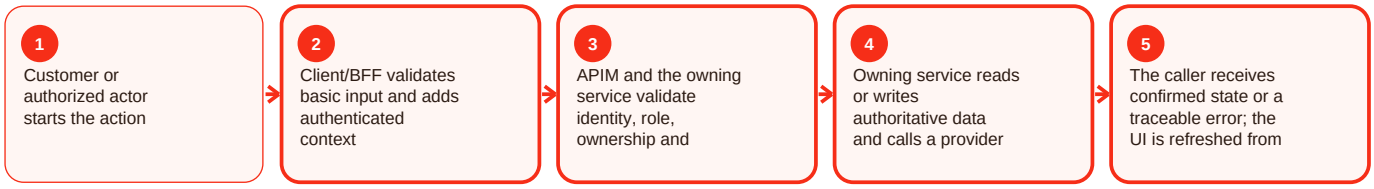
Summary

Refund / provider state is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For refund/provider state, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refund / provider state - Functional behavior and data

Current implementation

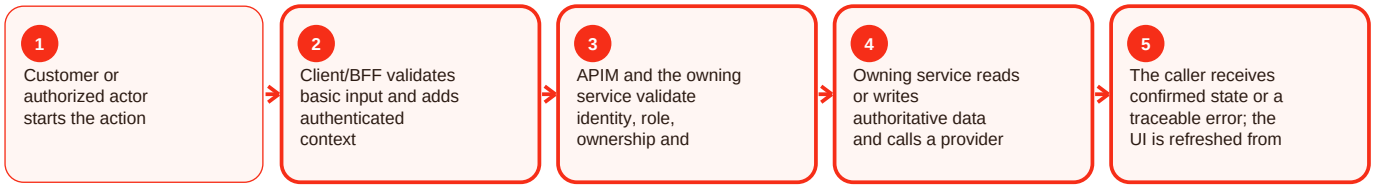
Summary

Refund / provider state is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For refund/provider state, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Refund / provider state - Operations, errors and deployment

Current implementation

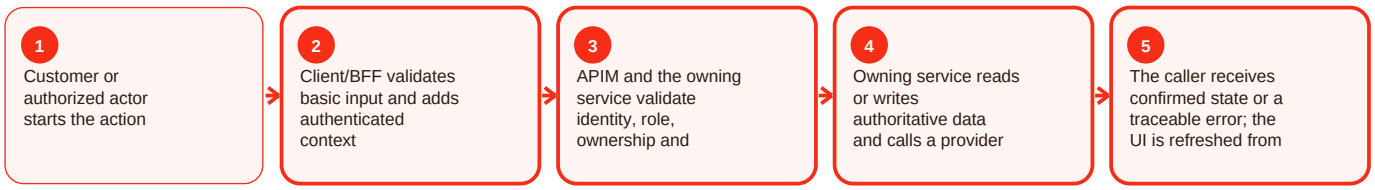
Summary

Refund / provider state is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For refund/provider state, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Azure communication services - Overview and responsibility

Current implementation

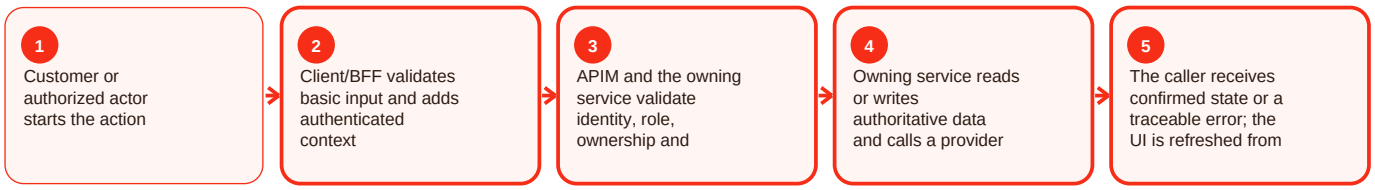
Summary

Azure communication services is part of the backend area of Craves. The active backend target is seven Spring Boot services: auth, user-chef, catalog, order, subscription, integration and notification. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Services own focused business domains, validate JWT role/ownership, expose health endpoints, evolve schema through Flyway where documented, and isolate provider-specific behavior behind integration/notification boundaries. For Azure Communication Services, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Service health is down: Database/config/dependency/startup failure. Resolution: Inspect health/readiness and recent deployment/config changes.

Request rejected: Validation, role, ownership or state rule failed. Resolution: Use stable error code/request ID and correct the actual failing rule.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/auth-service/README.md; services/user-chef-service/README.md; services/catalog-service/README.md; services/order-service/README.md; services/subscription-service/README.md; services/integration-service/README.md; services/notification-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Azure communication services - Functional behavior and data

Current implementation

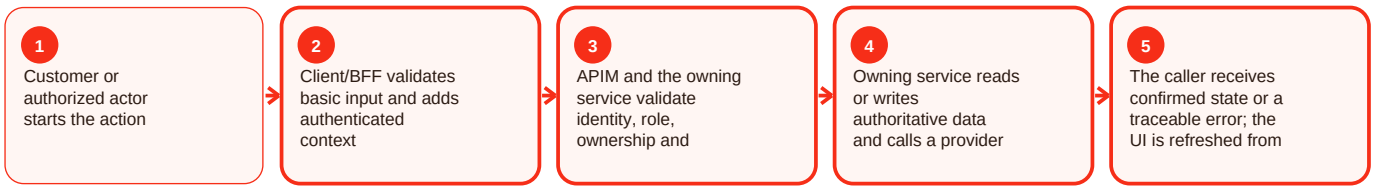
Summary

Azure communication services is part of the backend area of Craves. The active backend target is seven Spring Boot services: auth, user-chef, catalog, order, subscription, integration and notification. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Services own focused business domains, validate JWT role/ownership, expose health endpoints, evolve schema through Flyway where documented, and isolate provider-specific behavior behind integration/notification boundaries. For Azure Communication Services, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Service health is down: Database/config/dependency/startup failure. Resolution: Inspect health/readiness and recent deployment/config changes.

Request rejected: Validation, role, ownership or state rule failed. Resolution: Use stable error code/request ID and correct the actual failing rule.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/auth-service/README.md; services/user-chef-service/README.md; services/catalog-service/README.md; services/order-service/README.md; services/subscription-service/README.md; services/integration-service/README.md; services/notification-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Azure communication services - Operations, errors and deployment

Current implementation

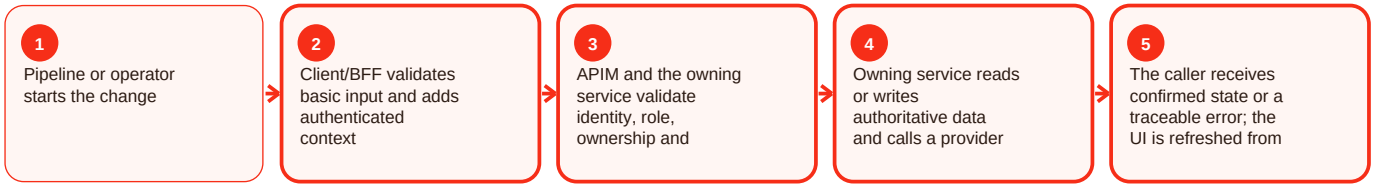
Summary

Azure communication services is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Azure Communication Services, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Fcm - Overview and responsibility

Current implementation

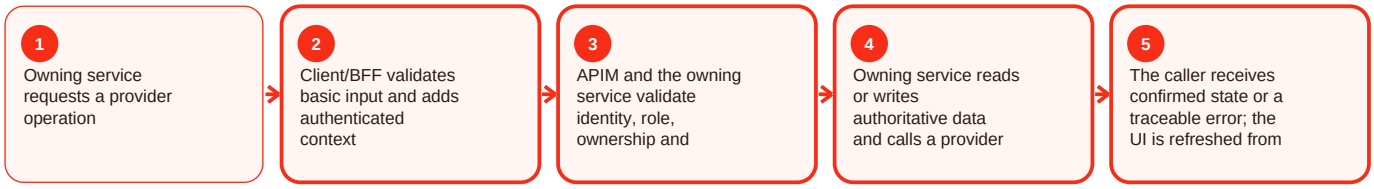
Summary

Fcm is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For FCM, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Fcm - Functional behavior and data

Current implementation

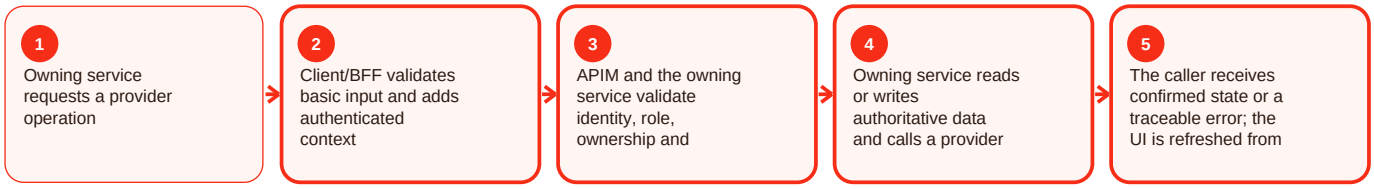
Summary

Fcm is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For FCM, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Fcm - Operations, errors and deployment

Current implementation

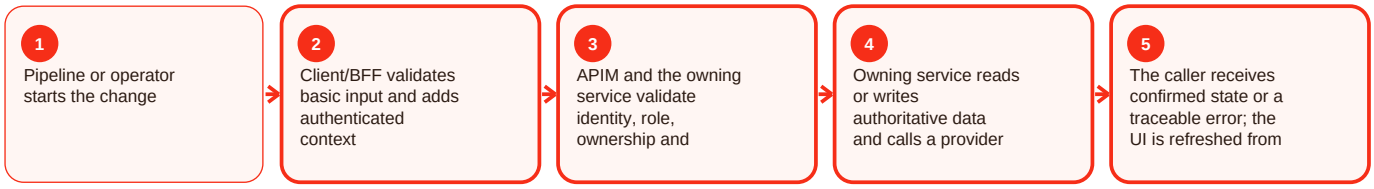
Summary

Fcm is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For FCM, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Service bus - Overview and responsibility

Current implementation

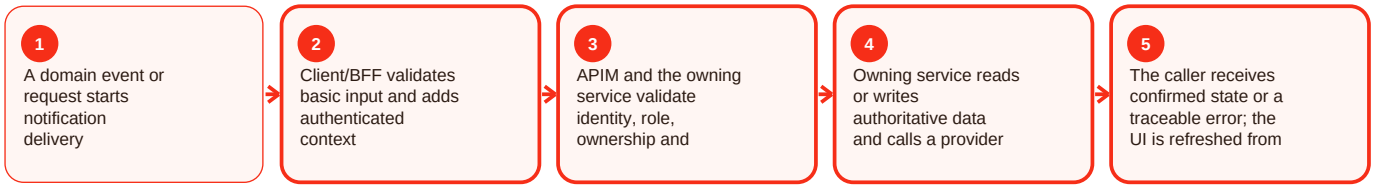
Summary

Service bus is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For Service Bus, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Service bus - Functional behavior and data

Current implementation

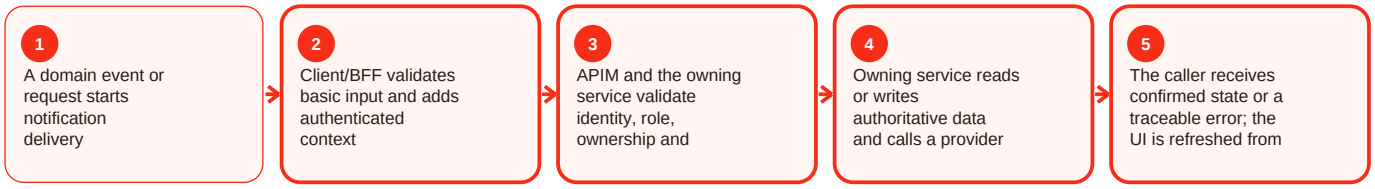
Summary

Service bus is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For Service Bus, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Service bus - Operations, errors and deployment

Current implementation

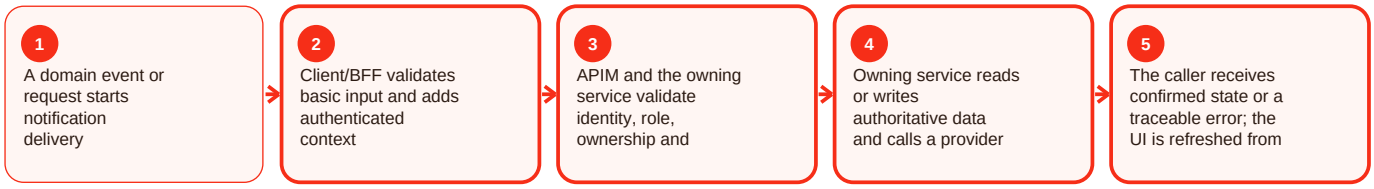
Summary

Service bus is part of the notification area of Craves. Notifications centralize email, SMS and push so domain services do not each implement vendor logic. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

notification-service documents Azure Communication Services, FCM, preferences, device tokens, optional Service Bus topic craves-domain-events, event-id idempotency, correlation propagation and a seven-day resend worker. For Service Bus, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

External channel stays pending: Provider adapter is disabled or unavailable. Resolution: Check channel configuration and delivery-attempt state.

Duplicate delivery: Retry did not reuse the event/request key. Resolution: Use idempotency evidence before resending.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/notification-service/README.md; apps/customer-web-next/src/components/notifications/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Event idempotency - Overview and responsibility

Current implementation

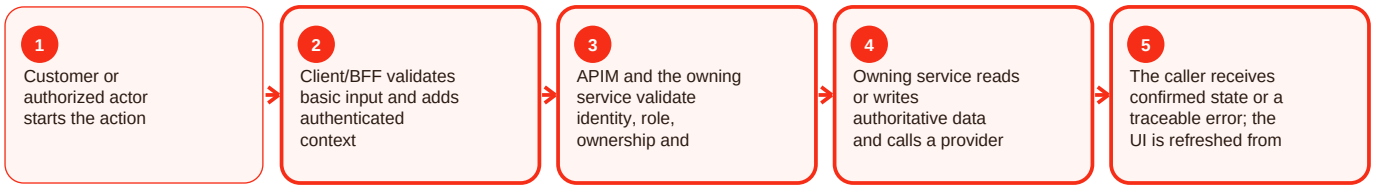
Summary

Event idempotency is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For event idempotency, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Event idempotency - Functional behavior and data

Current implementation

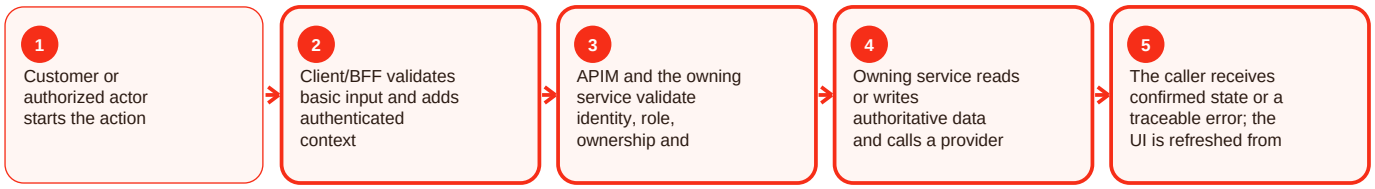
Summary

Event idempotency is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For event idempotency, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Event idempotency - Operations, errors and deployment

Current implementation

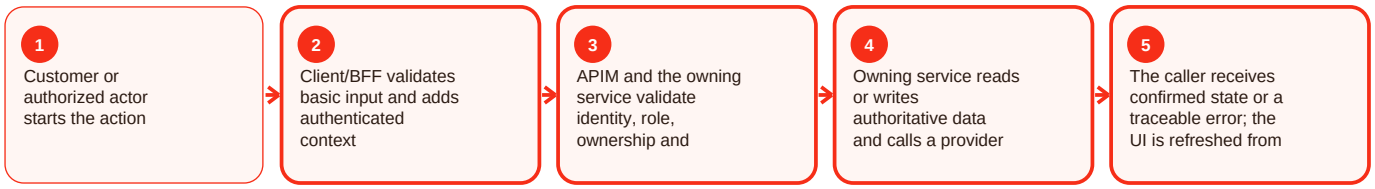
Summary

Event idempotency is part of the order area of Craves. Order management is the transactional spine of Craves: create, retrieve, status, cancellation/refund, delivery state, ETA/shortage, proof and administrative controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

order-service documents idempotency, structured Problem Details errors, reconciliation, provider integration and controlled multi-database routing. Supported roles include customer, chef, admin and delivery-partner contexts with ownership enforcement. For event idempotency, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Order not found: Wrong ID or ownership mismatch. Resolution: Verify ID and caller context; never expose another user’s order.

State change rejected: Transition is not valid from the current state. Resolution: Reload order state and use only the supported next action.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/order-service/README.md; apps/customer-web-next/src/components/order-history/; apps/customer-web-next/src/components/order-tracking/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Google maps - Overview and responsibility

Current implementation

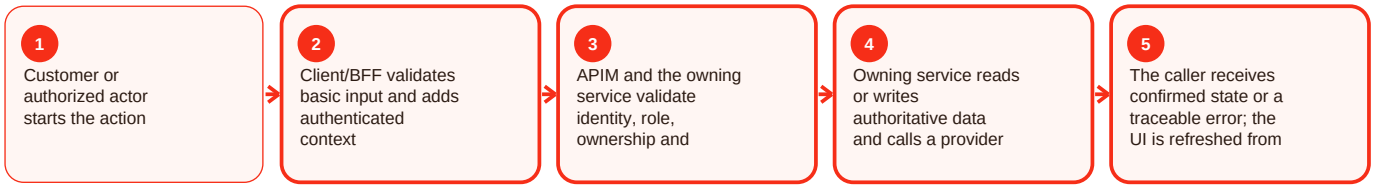
Summary

Google maps is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For Google Maps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Google maps - Functional behavior and data

Current implementation

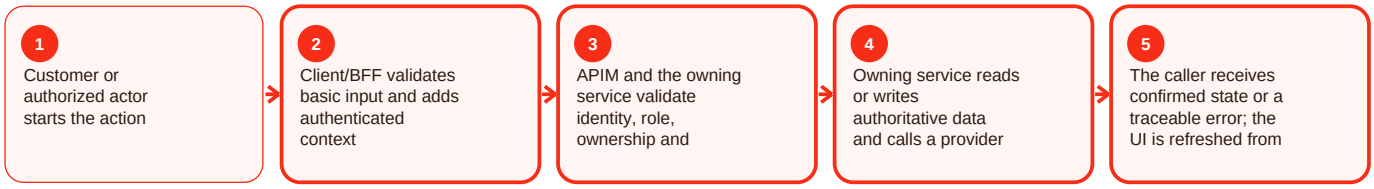
Summary

Google maps is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For Google Maps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Google maps - Operations, errors and deployment

Current implementation

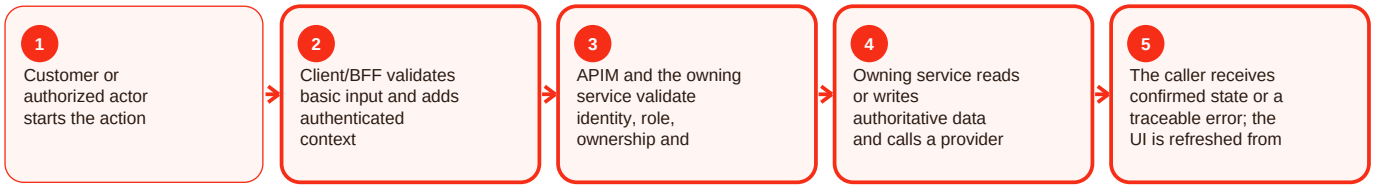
Summary

Google maps is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For Google Maps, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Google calendar - Overview and responsibility

Current implementation

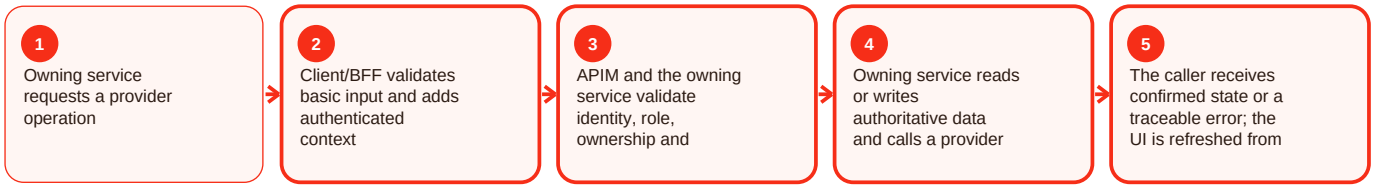
Summary

Google calendar is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For Google Calendar, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Google calendar - Functional behavior and data

Current implementation

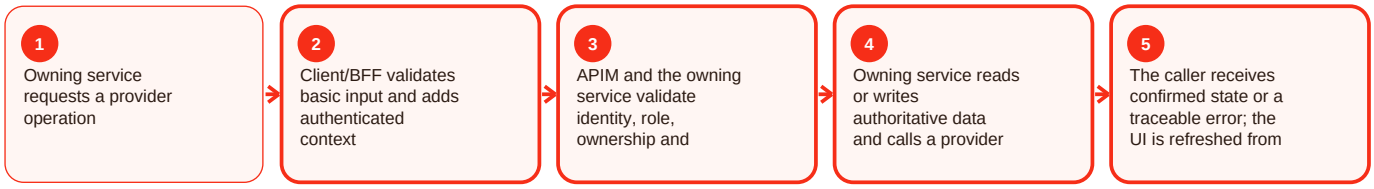
Summary

Google calendar is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For Google Calendar, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Google calendar - Operations, errors and deployment

Current implementation

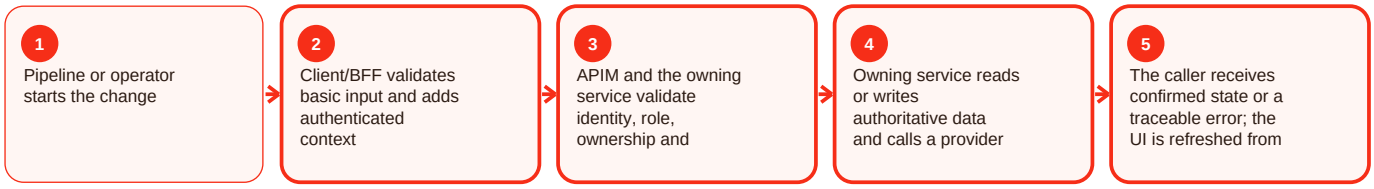
Summary

Google calendar is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Google Calendar, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Gmail drafts - Overview and responsibility

Current implementation

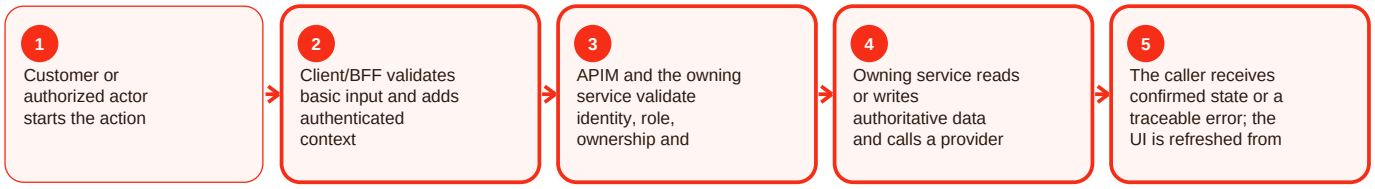
Summary

Gmail drafts is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Gmail drafts, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Gmail drafts - Functional behavior and data

Current implementation

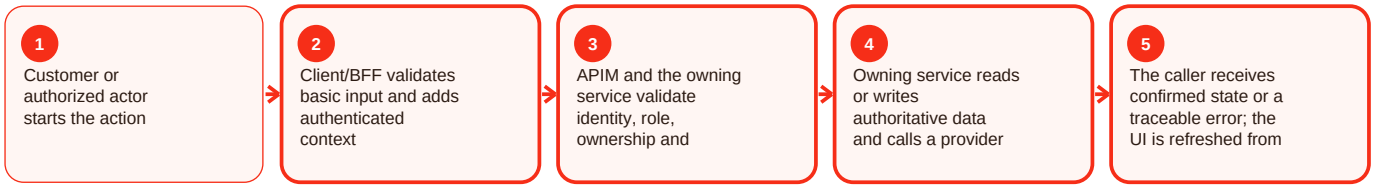
Summary

Gmail drafts is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Gmail drafts, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Gmail drafts - Operations, errors and deployment

Current implementation

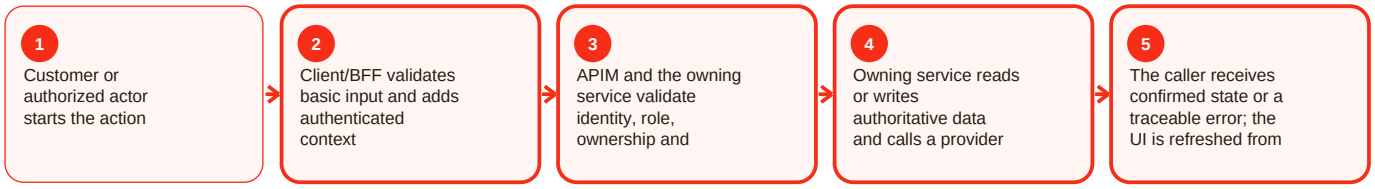
Summary

Gmail drafts is part of the ai area of Craves. Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions. For Gmail drafts, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

User grants - Overview and responsibility

Current implementation

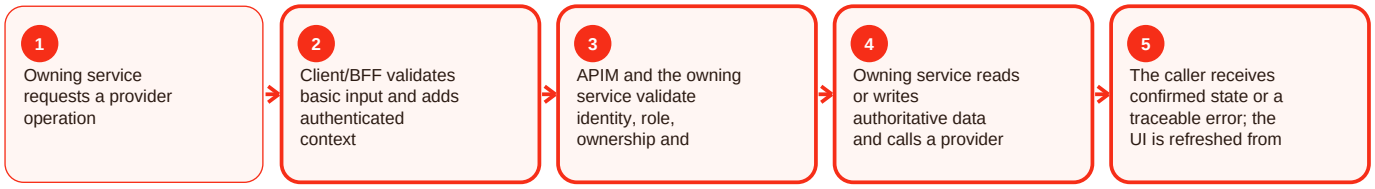
Summary

User grants is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For user grants, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

User grants - Functional behavior and data

Current implementation

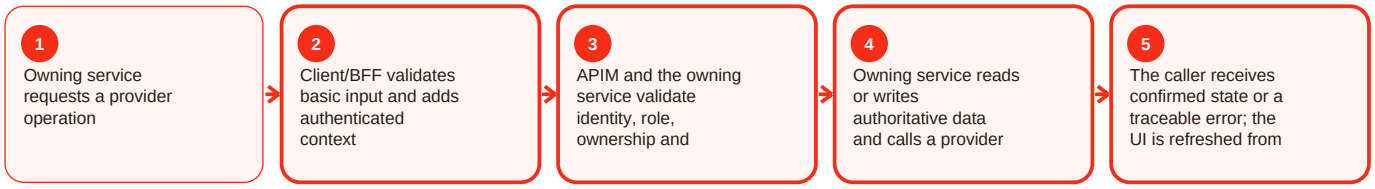
Summary

User grants is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For user grants, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

User grants - Operations, errors and deployment

Current implementation

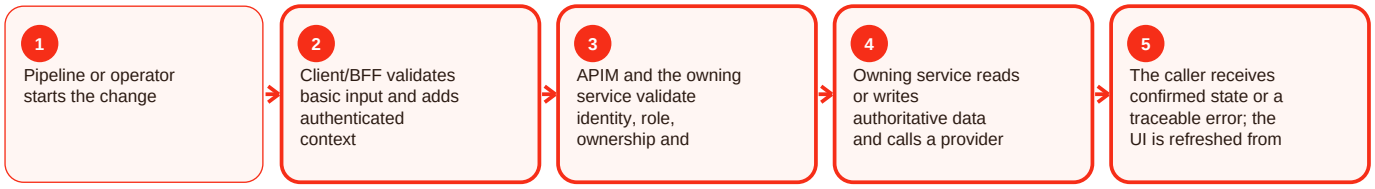
Summary

User grants is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For user grants, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pseudonymized audit - Overview and responsibility

Current implementation

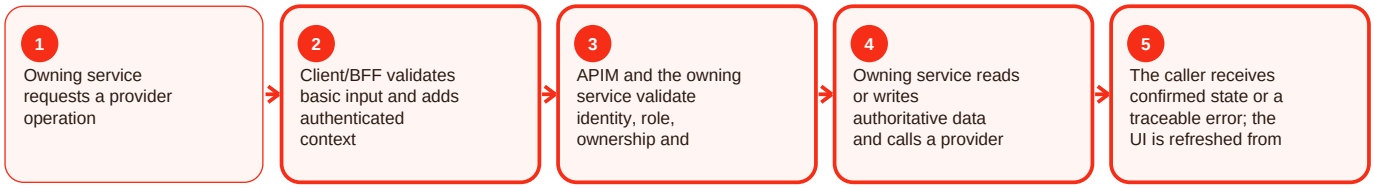
Summary

Pseudonymized audit is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For pseudonymized audit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pseudonymized audit - Functional behavior and data

Current implementation

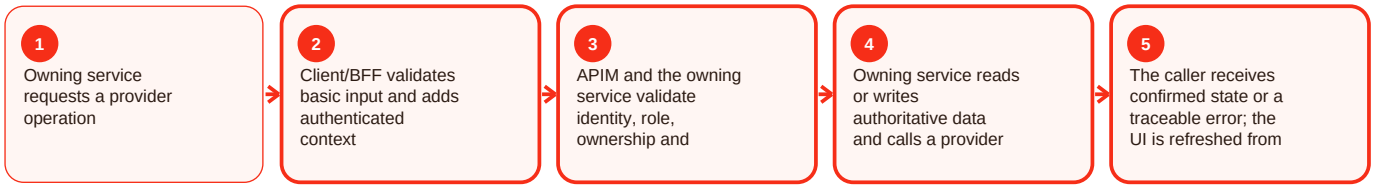
Summary

Pseudonymized audit is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For pseudonymized audit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Pseudonymized audit - Operations, errors and deployment

Current implementation

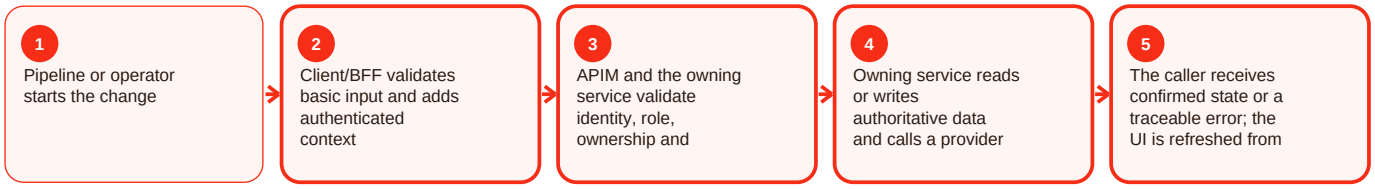
Summary

Pseudonymized audit is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For pseudonymized audit, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Blob / media trust - Overview and responsibility

Current implementation

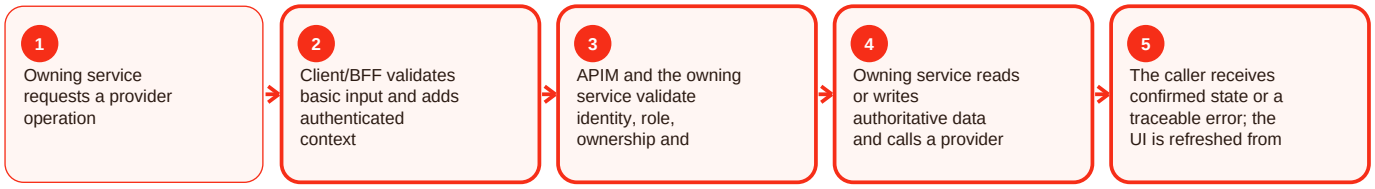
Summary

Blob / media trust is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For Blob/media trust, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Blob / media trust - Functional behavior and data

Current implementation

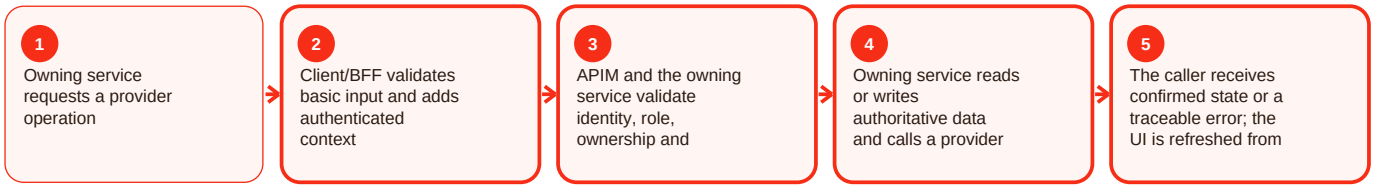
Summary

Blob / media trust is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For Blob/media trust, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Blob / media trust - Operations, errors and deployment

Current implementation

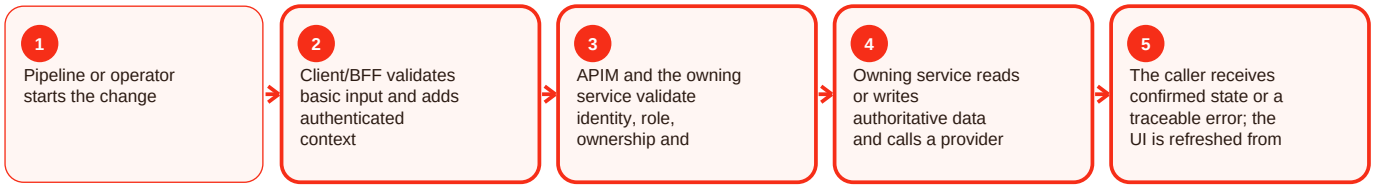
Summary

Blob / media trust is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Blob/media trust, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Firebase auth - Overview and responsibility

Current implementation

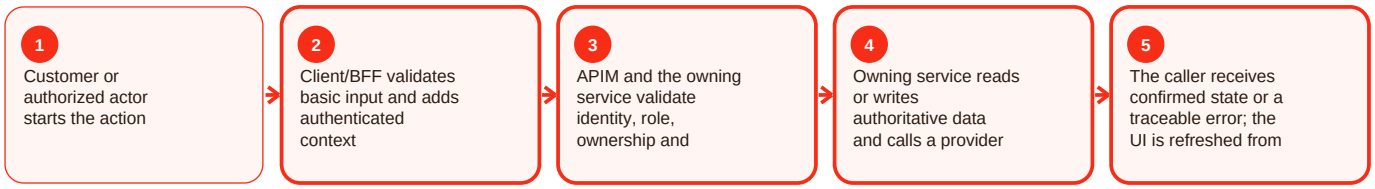
Summary

Firebase auth is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For Firebase auth, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Firebase auth - Functional behavior and data

Current implementation

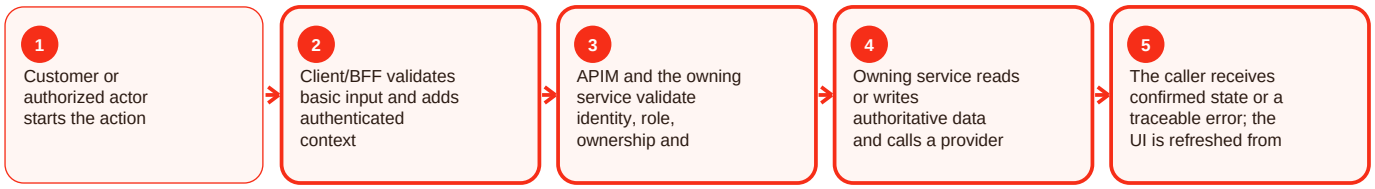
Summary

Firebase auth is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For Firebase auth, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Firebase auth - Operations, errors and deployment

Current implementation

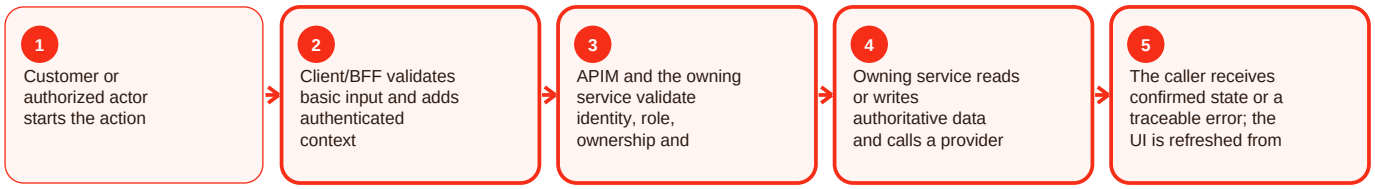
Summary

Firebase auth is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For Firebase auth, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra identity - Overview and responsibility

Current implementation

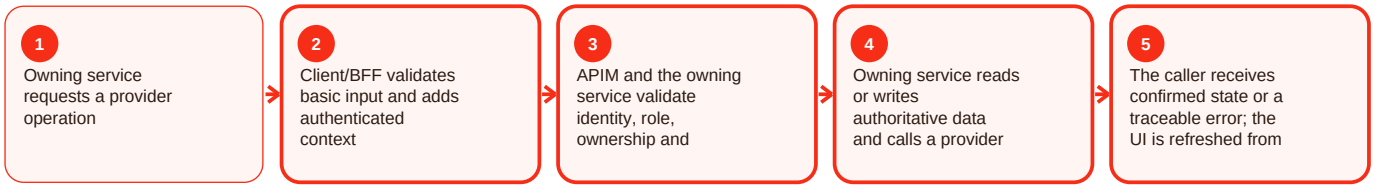
Summary

Entra identity is part of the integration area of Craves. integration-service is the provider boundary so third-party APIs do not leak provider-specific rules into every domain. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Repository documentation covers Google Maps fallback, Calendar creation, Gmail drafts, user grants, contact aliasing, pseudonymized audit, Razorpay requirements/payment links and deterministic disabled-by-default stubs. For Entra identity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Provider is unavailable: External dependency or credential/config problem. Resolution: Keep Craves state consistent and retry only when safe/idempotent.

Provider contract changed: Adapter no longer matches vendor response. Resolution: Fix and test the adapter rather than leaking vendor logic into domains.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/integration-service/README.md; services/integration-service/**/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.

Repository gaps that must stay visible

- APIM README cites infra/apim/craves-openapi.yaml, but that literal artifact was not resolvable on main during this evidence snapshot.
- customer-web-next README names azure-pipelines-customer-web-next.yml, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.